

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.8 · última da fase

Consolidação: o primeiro tijolo do Núcleo

O repositório vira portfólio, e a fronteira entre o que é conhecimento portátil e o que vira framework fica explícita.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 a 0.7 (tag **aula-0.7**)

1. Objetivo de aprendizagem

Ao final desta aula o seu repositório estará organizado a ponto de outra pessoa rodar o sistema seguindo só o README, e você será capaz de apontar, sem olhar o código, o que da Fase 0 é conhecimento portátil e o que a Fase 1 vai substituir por framework.

2. Por que esta aula importa

Nenhum mecanismo novo hoje. O trabalho é de arrumação, e ele importa por dois motivos que não têm nada a ver com elegância.

O primeiro é prático. Você vai levar este repositório para uma entrevista, para um cliente, ou vai voltar a ele daqui a três meses. Repositório que outra pessoa consegue rodar em dois minutos é portfólio. Repositório que só roda na sua máquina, porque falta um passo que você não escreveu, é exercício.

O segundo é intelectual, e é o mais importante da fase inteira. Na próxima aula você abre o LangGraph. Se atravessar essa fronteira sem saber exatamente o que está entregando ao framework, você vira usuário de framework. Se souber, você vira alguém que **escolhe** framework, e sabe dizer o que ganha e o que perde em cada escolha. A diferença entre as duas posições é a carreira inteira.

3. Teoria

3.1 O que você construiu

Vale olhar para trás com precisão, porque a lista é maior do que parece de dentro.

Peça	O que faz	Aula
Camada de provider	Uma função para dois SDKs, com os dialetos confinados	0.1
Ciclo de tool use	As quatro etapas, na mão, nos dois dialetos	0.2
Laço de agente	Encadeamento autônomo, com quatro condições de parada	0.3
Registro de ferramentas	Fonte única da verdade, schema derivado da função	0.4
Três ferramentas de domínio	CRM, agenda e tickets sobre SQLite	0.5
Resiliência	Erro de ferramenta como observação, não como morte	0.6
Saída estruturada	Contrato validado para consumo por outro sistema	0.6
Observabilidade	Árvore completa de execução, com custo e latência	0.7

Sem uma linha de framework de agente.

3.2 A fronteira: o que é portátil e o que vira framework

Esta seção é a mais importante da aula, e talvez da fase.

Conhecimento portátil, que você leva para qualquer stack:

- O ciclo de quatro etapas do tool use: você declara, o modelo pede, você executa, você devolve.

- A ideia de que **o agente é o laço**, não o modelo.
- As quatro condições de parada, e a noção de que bater o limite não é sucesso.
- A separação entre definição e execução de ferramenta, ligadas só pelo nome.
- Descrição de ferramenta é prompt, e muda a acurácia de seleção.
- Erro que o agente pode contornar vira observação; erro que ele não pode contornar sobe.
- Validação garante formato, nunca verdade.
- O vocabulário de trace, span e generation.

O que a Fase 1 substitui por framework: o `while` artesanal vira `StateGraph`; a lista de mensagens vira estado explícito e tipado; a persistência, que hoje simplesmente não existe, vira *checkpoint*; e a interrupção humana, hoje só um comentário no código, vira primitivo de verdade.

REPARE NO QUE NÃO ESTÁ NA SEGUNDA LISTA

O registro de ferramentas, o domínio, a resiliência e a observabilidade atravessam a fronteira **quase intactos**. Isso não é acaso: foi exatamente o motivo de construí-los desacoplados, aula após aula. Quando a Fase 1 trocar o laço, você vai reaproveitar tudo isso sem reescrever.

3.3 Motor e domínio: a fronteira que torna o Núcleo vendável

Há uma segunda fronteira, menos visível e igualmente decisiva. O **motor** (provider, laço, registro, resiliência, observabilidade) é agnóstico de domínio. O **domínio** (banco, ferramentas, instrução de sistema) é o que muda de cliente para cliente.

Se importar o pacote registrasse automaticamente as ferramentas do FlowDesk, o motor carregaria o domínio junto, e replicar em outro cliente significaria arrancar código. Como a separação existe, replicar é plugar outra base e outras ferramentas.

Fronteira de arquitetura que não tem teste é fronteira que alguém apaga numa refatoração distraída daqui a dois meses. Por isso ela ganha um teste hoje.

3.4 Dois tipos de teste que respondem perguntas diferentes

Até agora cada aula testou a sua peça. Faltam dois tipos de teste que só fazem sentido no fim.

O **teste de fumaça** não testa comportamento: testa que o pacote se monta e que a API pública está de pé. É ele que pega o import quebrado depois de uma refatoração, e custa dez linhas.

O **teste de ponta a ponta** responde a pergunta que ninguém tinha feito: as cinco camadas funcionam **juntas**? Cada aula provou a sua peça isoladamente; integração é outra coisa.

3.5 README é interface, não documentação

O README é a única parte do repositório que a maioria das pessoas vai ler. Cinco seções bastam: o que o sistema faz em duas frases; como rodar do zero; a arquitetura; como rodar os testes; e o mapa de tags por aula.

E uma sexta que quase ninguém escreve: **o que o sistema ainda não faz**. Um README honesto sobre os limites vale mais, para quem avalia, do que um que promete demais. Quem lê "não tem persistência, não tem aprovação humana, não está no ar" entende que você sabe onde está. Quem lê promessa demais confere e descobre.

4. Mão na massa

Passo 1 — Declarar a API pública

O que e por quê: até aqui o `__init__.py` estava vazio e tudo era importado pelo caminho interno. Declarar o que é público é dizer onde estão as portas da casa.

```
"""Nucleo de Agentes Operacionais - Fase 0.

Um agente de acao em codigo puro, sem framework: agnostico de provider,
com registro de ferramentas, resiliencia a erro e observabilidade.

Este arquivo declara a API PUBLICA do pacote. Ate a Aula 0.7 ele estava
vazio, e tudo era importado pelo caminho interno. Declarar o que e
publico e dizer, para voce mesmo daqui a tres meses e para quem clonar
o repositorio, onde estao as portas da casa.

Uso minimo:

    from nucleo import banco, ferramentas          # registra as ferramentas
    from nucleo import rodar_agente, schemas, executores_seguros

    banco.criar_schema()
    exe = rodar_agente("A Ana relatou erro de login.",
                      schemas(), executores_seguros())
    print(exe.resposta)
"""

# A ordem dos imports abaixo segue as camadas do sistema, de baixo para
# cima. Ela e, na pratica, o mapa da Fase 0.

# Camada 1 - falar com o modelo (Aulas 0.1 e 0.3)
from nucleo.config import ANTHROPIC_MODEL, OPENAI_MODEL
from nucleo.providers import (PedidoFerramenta, RespostaLLM, chat,
                             chat_ferramentas, mensagens_de_resultado)

# Camada 2 - o laco (Aula 0.3)
from nucleo.agente import Execucao, rodar_agente

# Camada 3 - ferramentas: definicao, execucao e dominio (Aulas 0.4 e 0.5)
from nucleo.registro import (Ferramenta, executar_ferramenta, executores,
                             ferramenta, schemas, schemas_para)

# Camada 4 - contratos e resiliencia (Aula 0.6)
from nucleo.modelos import QualificacaoLead
from nucleo.estruturado import extrair_estruturado
from nucleo.resiliencia import (ErroIrrecuperavel, executores_seguros,
                                proteger)

# Camada 5 - observabilidade (Aula 0.7)
from nucleo.observabilidade import (arvore, iniciar_trace, observar,
                                    registrar_uso, resumo)

# ferramentas e banco NAO sao importados aqui de proposito: importar
# nucleo.ferramentas tem efeito colateral (registra as tres ferramentas
# do dominio no REGISTRO). Quem quer o dominio pede por ele
# explicitamente. O motor e o dominio ficam separados, e e isso que
# torna o Nucleo replicavel em outro cliente.
```

O que faz: a ordem dos imports segue as camadas do sistema, de baixo para cima, então o arquivo é, na prática, o mapa da Fase 0. O `__all__` fecha a lista.

E note o comentário final, que é a decisão de arquitetura da seção 3.3 virando código: `ferramentas` e `banco` **não** são importados aqui, porque importar `nucleo.ferramentas` tem efeito colateral.

Passo 2 — O teste de fumaça, e o que ele protege

```
# ----- FUMACA -----

def test_api_publica_esta_de_pe():
    """Todo simbolo anunciado em __all__ existe de verdade."""
    faltando = [n for n in nucleo.__all__ if not hasattr(nucleo, n)]
    assert faltando == []

def test_todos_os_modulos_importam():
    """Pega o import circular ou quebrado depois de uma refatoracao."""
    from nucleo import (agente, banco, config, estruturado, ferramentas,
                        modelos, observabilidade, providers, registro,
                        resiliencia)
    assert all([agente, banco, config, estruturado, ferramentas, modelos,
                observabilidade, providers, registro, resiliencia])

def test_motor_e_dominio_sao_separados():
    """Importar o pacote NAO registra as ferramentas do FlowDesk.

    E isso que torna o Nucleo replicavel: outro cliente pluga o proprio
    dominio sem herdar o nosso. Se este teste quebrar, alguem importou
    nucleo.ferramentas dentro do __init__.
    """
    import importlib
    import sys

    for mod in ["nucleo", "nucleo.registro", "nucleo.ferramentas"]:
        sys.modules.pop(mod, None)

    limpo = importlib.import_module("nucleo")
    assert limpo.schemas()["anthropic"] == []

    # Agora sim, pedindo o dominio explicitamente:
    importlib.import_module("nucleo.ferramentas")
    nomes = {s["name"] for s in limpo.schemas()["anthropic"]}
    assert nomes == {"buscar_atualizar_crm", "agendar_reuniao",
                     "abrir_ticket"}
```

O que faz: os dois primeiros testes custam quase nada e pegam a classe de bug mais chata de diagnosticar. O terceiro é diferente: ele **protege uma decisão de arquitetura**. Se alguém, no futuro, importar o domínio dentro do `__init__` por conveniência, o teste falha e explica por quê.

Passo 3 — O teste de ponta a ponta

O que e por quê: o fluxo completo, com um erro no meio do caminho, exercitando as cinco camadas de uma vez.

```
def test_fluxo_completo_com_recuperacao(ambiente, monkeypatch):
    """As cinco camadas juntas, com um erro no meio do caminho.

    O modelo erra a prioridade (fora do enum), le o erro como
    observacao, corrige e conclui. Tudo isso rastreado.
    """
```

```

turno = {"role": "assistant", "content": []}
_rroteiro(monkeypatch, [
    RespostaLLM("", [PedidoFerramenta(
        "1", "buscar_atualizar_crm", {"email": "ana@techflow.com"})],
        turno, tokens_entrada=1180, tokens_saida=62),
    RespostaLLM("", [PedidoFerramenta(
        "2", "abrir_ticket", {"email": "ana@techflow.com",
            "assunto": "Erro no login",
            "descricao": "Erro 500 no SSO",
            "prioridade": "urgentissima"})],
        turno, tokens_entrada=1420, tokens_saida=88),
    RespostaLLM("", [PedidoFerramenta(
        "3", "abrir_ticket", {"email": "ana@techflow.com",
            "assunto": "Erro no login",
            "descricao": "Erro 500 no SSO",
            "prioridade": "alta"})],
        turno, tokens_entrada=1690, tokens_saida=85),
    RespostaLLM("Ticket #1 aberto com prioridade alta.", [], None,
        tokens_entrada=1810, tokens_saida=31),
])

exe = nucleo.rodar_agente(
    "A Ana do TechFlow relatou erro 500 no login.",
    nucleo.schemas(), nucleo.executores_seguros(), verbose=False)

# 1. O agente concluiu, apesar do erro no meio.
assert exe.parou_por == "resposta_final"
assert exe.iteracoes == 4

# 2. O efeito colateral aconteceu de verdade, no banco.
con = banco.conectar()
tickets = con.execute(
    "SELECT prioridade FROM tickets").fetchall()
con.close()
assert len(tickets) == 1          # o invalido NAO gravou
assert tickets[0]["prioridade"] == "alta"

# 3. A execucao inteira ficou rastreada.
r = nucleo.resumo(exe.trace)
assert r["chamadas_ao_modelo"] == 4
assert r["ferramentas_executadas"] == 3
assert r["tokens_entrada"] == 6100    # a soma das quatro voltas

```

O que faz: o modelo erra a prioridade, lê o erro como observação, corrige e conclui. As três asserções verificam coisas diferentes: que o agente concluiu apesar do erro, que o efeito colateral aconteceu **no banco** (e que a tentativa inválida não gravou nada), e que a execução inteira ficou rastreada com o custo somado das quatro voltas.

Passo 4 — A demo que fica no repositório

O que e por quê: desde a aula 0.1 o código de demonstração era descartável, em `scratch/`. Este é diferente, e a exceção é consciente: é a documentação executável do projeto, a porta de entrada de quem clona e o roteiro da demo gravada. Por isso vive em `exemplos/` e é versionado.

```

# A politica comercial mora AQUI, no prompt, nao nas ferramentas.
# Trocar de cliente e trocar este texto e a base, nao o codigo.
SISTEMA = """Voce e o assistente comercial do FlowDesk, um SaaS B2B.

Consulte o CRM antes de qualquer acao sobre um lead. Considere
qualificado quem tem plano pro ou enterprise, ou MRR acima de 300;
nesse caso, atualize o status e proponha uma reuniao. Problema tecnico

```

```
vira ticket, nunca reuniao."""
```

```
PERGUNTA = ("A Ana do TechFlow escreveu: o login pelo SSO esta dando "  
            "erro 500 desde ontem, e ela quer conversar sobre migrar "  
            "de plano.")
```

O que faz: repare onde mora a política comercial. Trocar de cliente é trocar este texto e a base, não o código. É a regra da aula 0.5 no lugar mais visível do repositório.

O resto do arquivo imprime, em quatro blocos: a resposta, a árvore da execução, o custo da sessão e o contrato estruturado. Ou seja, as aulas 0.3, 0.7 e 0.6 numa tela só.

Passo 5 — O README

O que e por quê: seis seções, incluindo a dos limites. O repositório traz o arquivo completo; a estrutura que ele documenta é esta:

```
nucleo/  
├─ __init__.py          a API publica do pacote  
├─ config.py           nomes de modelo, lidos do .env      (0.1)  
├─ providers.py        a camada agnostica de provider      (0.1, 0.3, 0.7)  
├─ agente.py           o laco: decisao, acao, observacao  (0.3, 0.7)  
├─ registro.py         o decorator que gera os schemas  (0.4, 0.5)  
├─ banco.py            o CRM mockado em SQLite          (0.5)  
├─ ferramentas.py      as tres ferramentas do dominio      (0.5)  
├─ modelos.py          os contratos Pydantic            (0.6)  
├─ resiliencia.py       erro de ferramenta como observacao (0.6)  
├─ estruturado.py      a saida final validada          (0.6)  
└─ observabilidade.py  trace, span e generation          (0.7)
```

Passo 6 — Fechar a fase

```
pytest -v          # esperado: 58 passed, 2 deselected  
python exemplos/qualificar_lead.py  
  
git add .  
git commit -m "Aula 0.8: consolidacao da Fase 0"  
git tag aula-0.8  
git tag -a fase-0 -m "Fase 0 completa: agente em codigo puro"  
git push && git push --tags
```

Duas tags de propósito: `aula-0.8` mantém o padrão da série, e `fase-0` é a tag anotada que marca o marco. É nela que a Fase 1 vai se apoiar.

5. Portabilidade

Esta aula inteira é sobre portabilidade, então a seção aqui é outra: **o que levar para o próximo projeto**, independente de agente.

Declarar API pública num `__init__.py`, separar motor de domínio e proteger a separação com teste, teste de fumaça barato, exemplo executável versionado, README que admite limites. Nada disso é sobre IA. É higiene de projeto, e é o que faz um repositório sobreviver ao contato com outra pessoa.

6. Checkpoint da Fase 0

- O repositório está organizado e outra pessoa roda o sistema seguindo só o README.
- O sistema roda com Claude ou com OpenAI por configuração, não por edição de código.
- `pytest` mostra 58 passed, 2 deselected, sem rede.
- Você explica o laço sem olhar o código, e aponta a fronteira da seção 3.2.

Cumpridos os quatro, a Fase 0 está fechada. O primeiro tijolo do Núcleo está de pé.

7. Erros comuns e troubleshooting

- **ImportError circular ao declarar o `__init__`.** Algum módulo importa `nucleo` em vez de `nucleo.alguma_coisa`. Dentro do pacote, sempre importe o módulo específico.
- **O teste de separação motor/domínio falha.** Alguém importou `nucleo.ferramentas` no `__init__`. É exatamente o que o teste existe para pegar.
- **A demo falha com erro de chave.** O `exemplos/` precisa do `.env` real. Os testes não, porque o `conftest.py` injeta chaves falsas.
- **O README promete o que o sistema não faz.** Mais comum do que parece, e caro: quem avalia confere.
- **A tag `fase-0` não aparece no remoto.** Faltou o `git push --tags`.

8. Ponte para a Fase 1

Seu agente funciona e morre. Se o processo cair no meio de um atendimento, tudo se perde: a lista de mensagens estava na memória, e memória não sobrevive a *restart*. Não há como pausar antes de uma ação sensível e pedir aprovação. Não há como retomar do ponto exato.

Essas três coisas, persistência, interrupção e retomada, são exatamente o que um grafo de estado resolve. A **Fase 1** começa migrando o seu laço para o LangGraph, e a primeira coisa que você vai fazer é reconhecer, no estado do grafo, a lista de mensagens que você já carregava na mão.